

Processor Verification - Lab 4

System Verilog Assertions

Lorenzo Pattaro Zonta

May 3, 2026

1 Fetch Unit

1.1 Functional Description

The fetch unit maintains a program counter (PC) and outputs the instruction located at the current PC address. It supports normal sequential execution and conditional branching via an alternate branch address.

1.2 Module Interface

Signal	Direction	Width	Description
clk	input	1	Clock signal.
rst_n	input	1	Active-low synchronous reset.
pc_en	input	1	Enables PC increment (sequential fetch).
branch_en	input	1	Enables branch to branch_addr.
branch_addr	input	8	Branch target address.
instr	output	16	Instruction word fetched from memory.

1.3 Behavioral Description

An instruction memory is initialized with dummy values. On each clock edge, the PC updates to either the branch address (if branch is enabled), the next sequential address ($PC + 2$ if `pc_en` is asserted), or holds its value. The output instruction is read from memory at the updated PC. On reset, the PC is cleared to zero and the instruction at address zero is output.

1.4 List of assertions

The following is the list of the assertions for the fetch unit. A thing to be noted is that to directly access the program counter I have used `dut.pc`, as well as `dut.mem`, as they are neither a input nor an output of the device under test. To signal to the user that an assertion has failed, I have used `$error` because these are design bugs that require investigation. Moreover I added value of the error outputs to easily debug. Except for PT01, I have added `disable iff (!rst_n)` to exclude assertion that must not be checked with the reset. This instruction it is also needed because it has to be considered the first reset edge, when `pc` and `instr` signals are still X (unknown), and it can lead to false negative.

- Reset: after a reset cycle, PC must be zero.

```
property PT01_reset_pc_zero;
    @(posedge clk) !rst_n | => (dut.pc == 8'h00);
endproperty

assert property (PT01_reset_pc_zero)
else $error("On reset PC hasn't gone to zero: pc=%0h", dut.pc);
```

- If `branch_en` is asserted, PC must load `branch_addr` regardless of `pc_en`.

```
property PT02_branch_precedence_over_pc_en;
  @(posedge clk) disable iff (!rst_n)
    branch_en | => (dut.pc == $past(branch_addr));
endproperty
```

```
assert property (PT02_precedence_over_pc_en)
else $error("Branch priority load failed");
```

- If `pc_en` is asserted (and no branch), PC must increment by 2.

```
property PT03_pc_increment;
  @(posedge clk) disable iff (!rst_n)
    (pc_en && !branch_en) | => (dut.pc == ($past(dut.pc) + 8'd2));
endproperty
```

```
assert property (PT03_pc_increment)
else $error("PC hasn't incremented by 2 on pc_en && !branch_en. pc = %0h, Past pc = %0h",
  ↪ dut.pc, $past(dut.pc));
```

- If neither `branch_en` nor `pc_en` is asserted, PC must hold its value.

```
property PT04_pc_stability;
  @(posedge clk) disable iff (!rst_n)
    (!pc_en && !branch_en) | => (dut.pc == $past(dut.pc));
endproperty
```

```
assert property (PT04_pc_stability)
else $error("PC hasn't hold its value on !pc_en && !branch_en. pc = %0h, Past pc = %0h",
  ↪ dut.pc, $past(dut.pc));
```

- `instr` must always equal `mem[PC]`.

```
property PT05_instruction_consistency;
  @(posedge clk) disable iff (!rst_n)
    (instr == dut.mem[dut.pc]);
endproperty
```

```
assert property (PT05_instruction_consistency)
else $error("instruction isn't equal to mem[pc]. Instr = %0h, Mem[pc] = %0h", instr,
  ↪ dut.mem[dut.pc]);
```

- PC must never go out of memory bounds (0-255) or be equal to unknowns or high impedance values.

```
property PT06_pc_range_safety;
  @(posedge clk) disable iff (!rst_n)
    (dut.pc inside {[8'h00:8'hFF]}) && !$isunknown(dut.pc);
endproperty
```

```
assert property (PT06_pc_range_safety)
else $error("PC out of range. PC = %0h", dut.pc);
```

2 Decode Unit

2.1 Functional Description

The decode unit receives a 16-bit instruction and splits it into opcode, destination register, source register, and immediate fields. It models a fixed two-cycle decode latency and performs a simple data hazard check that can stall decoding.

2.2 Module Interface

Signal	Direction	Width	Description
clk	input	1	Clock signal.
rst_n	input	1	Active-low synchronous reset.
instr_valid	input	1	Indicates the input instruction is valid.
instr	input	16	Instruction word from fetch stage.
decode_done	output	1	Pulses when decode latency completes.
opcode	output	4	Opcode field (bits 15:12).
rd	output	4	Destination register (bits 11:8).
rs	output	4	Source register (bits 7:4).
imm	output	4	Immediate field (bits 3:0).
hazard_stall	output	1	High when a data hazard is detected.

2.3 Behavioral Description

Instruction acceptance occurs when `instr_valid=1` and `hazard_stall=0` in the next cycle. On acceptance, the fields `opcode`, `rd`, `rs`, and `imm` capture `instr[15:12]`, `instr[11:8]`, `instr[7:4]`, and `instr[3:0]`, respectively. After an instruction is accepted in cycle N , `decode_done` must assert exactly in cycle $N + 2$ and it must be a single-cycle pulse.

A hazard is detected when the destination register `rd` of the previously accepted instruction matches the source register `rs` of the current instruction. When this hazard occurs, `hazard_stall` asserts, the current instruction is not accepted until the stall clears, and `decode_done` must not assert during the stall window. While `instr_valid=1` and `hazard_stall=1`, the decoded fields (`opcode`, `rd`, `rs`, `imm`) must retain their previous values and only update upon acceptance.

For back-to-back valid instructions with no hazard, each instruction is accepted immediately in its cycle, and each acceptance produces a `decode_done` pulse two cycles later, which may result in consecutive `decode_done` pulses.

When `rst_n=0`, all outputs are cleared to zero, and no instruction is accepted or completed until reset is released.

2.4 List of assertions

The following is the list of assertions for the decode unit. For the hazard detected properties (PT04, PT05 and PT06) I have decided to write three different properties to have an higher granularity for debugging.

- An instruction is accepted when `instr_valid=1` and `hazard_stall=0` in the next cycle. On acceptance, the fields (`opcode`, `rd`, `rs`, `imm`) must capture the corresponding bits of `instr`.

```
property PT01_instruction_acceptance;
  @(posedge clk) disable iff (!rst_n)
    instr_valid == 1'b1 |> !hazard_stall |> (opcode == $past(instr[15:12])) &&
                                             (rd == $past(instr[11:8])) &&
                                             (rs == $past(instr[7:4])) &&
                                             (imm == $past(instr[3:0]));

endproperty

assert property (PT01_instruction_acceptance)
else $error("PT01: instruction hasn't been accepted or (correctly) filled with instr when
↳ instr_valid and no hazard stall");
```

- After acceptance in cycle N , `decode_done` must assert exactly in cycle $N+2$.

```
property PT02_decode_latency;
  @(posedge clk) disable iff (!rst_n)
    instr_valid == 1'b1 |> !hazard_stall |> ##2 decode_done == 1'b1;

endproperty
```

```

assert property (PT02_decode_latency)
else $error("PT02: decode_done hasn't been asserted in cycle N+2 after acceptance");

```

- decode_done must be a single cycle pulse.

```

property PT03_decode_single_cycle;
  @(posedge clk) disable iff (!rst_n)
    decode_done == 1'b1 | => decode_done == 1'b0;
endproperty

assert property (PT03_decode_single_cycle)
else $error("PT03: decode_done hasn't endured for only a single cycle");

```

- A hazard occurs when the destination register (rd) of the previously accepted instruction matches the source register (rs) of the current instruction. When hazard is detected, hazard_stall must assert. Here I have written rd == instr[7:4] as rd contains the destination register of the previously accepted instruction and the current instruction source register (rs) will be filled with instr[7:4].

```

property PT04_hazard_occurrence;
  @(posedge clk) disable iff (!rst_n)
    (rd == instr[7:4]) && instr_valid | => hazard_stall == 1'b1;
endproperty

assert property (PT04_hazard_occurrence)
else $error("PT04: hazard_stall hasn't been asserted when rs matches previous rd");

```

- When hazard is detected, the current instruction is not accepted until the stall clears. Here and in the next property I have not checked the condition (rd == instr[7:4]) && instr_valid but hazard_stall in the same cycle of decode_done because I have to cover all the stall window and not only the cycle in which I detect an hazard. This property also cover the case in which both hazard_stall and instr_valid are both 1 and the decoded fields (opcode, rd, rs, imm) must retain their previous values and only update when the instruction is accepted.

```

property PT05_hazard_current_instruction;
  @(posedge clk) disable iff (!rst_n)
    hazard_stall == 1'b1 | ->
      (opcode == $past(opcode)) &&
      (rd == $past(rd)) &&
      (rs == $past(rs)) &&
      (imm == $past(imm));
endproperty

assert property (PT05_hazard_current_instruction)
else $error("PT05: current instruction fields haven't been blocked from being accepted
↳ when hazard_stall is asserted");

```

- When hazard is detected, decode_done must not assert during the stall window.

```

property PT06_hazard_decode_done;
  @(posedge clk) disable iff (!rst_n)
    hazard_stall == 1'b1 | -> decode_done == 1'b0;
endproperty

assert property (PT06_hazard_decode_done)
else $error("PT06: decode_done has been asserted when hazard occurs");

```

- When rst_n=0: all outputs must be cleared to zero; no instruction is accepted or completed until reset is released.

```

property PT07_reset_behaviour;
    @(posedge clk) !rst_n | => (decode_done == 1'b0)    &&
                                (opcode    == 4'h0)    &&
                                (rd        == 4'h0)    &&
                                (rs        == 4'h0)    &&
                                (imm       == 4'h0)    &&
                                (hazard_stall == 1'b0);

endproperty

assert property (PT07_reset_behaviour)
else $error("PT07: On reset, outputs haven't been initialized to zero or decode_done, or
↪ hazard_stall haven't been deasserted");

```

3 Appendix

3.1 Number of hours required

For the completion of this laboratory, it has been needed 13 hours, that comprehend laboratory, understanding of the assignment, coding assertions and writing the report.

3.2 Use of AI

Code review and syntax check.